Nom et Prénom : .... **CIN :**	**N° d'examen :** ..... **Code matière :** `code_matière` **Code sujet :** `code_sujet`	**réserver à la data matrix d'étudiant**

✂ - - -

ÉTUDE DE CAS : INTERNET DES OBJETS

L'Internet des objets (*en anglais* ***Internet of Things*** *ou* ***IoT***) est l'interconnexion via Internet des machines et des équipements intelligents. Cette technologie consiste à commander et/ou à récupérer des données à distance des équipements.

L'interconnexion des objets est gérée par un serveur nommé **« Broker ».** Les **Devices** connectés peuvent être des **« Publishers »** ou des **« Subscribers »**. Le protocole MQTT (**M**essage **Q**ueuing **T**elemetry **T**ransport) est utilisé pour la communication entre ces équipements. La figure 1 donne un exemple de schéma de connexion entre les éléments cités précédemment :

Figure 1 : Réseau regroupant un Broker et des Devices connectés

Au sein d'une entreprise, on veut mettre en œuvre cette technologie en créant trois espaces :

- **Espace client** : Une plateforme Web qui permet au client de visualiser les messages des divers **Devices**.
- **Espace Administrateur** : Une Application Console qui permet d'ajouter de nouveaux **Devices**.
- **Espace Technicien** : Application Desktop qui permet de planifier un plan de configuration des divers **Devices**.

La persistance des données est assurée par l'utilisation de serveurs de bases de données. Les deux figures 2 et 3 illustrent respectivement les espaces de travail, les acteurs humains et leurs interactions avec les diverses applications.

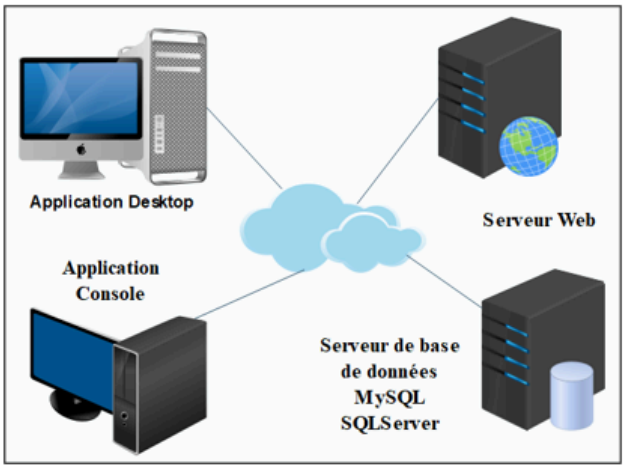

Figure 2 : Schéma structurel de service de l'IOT

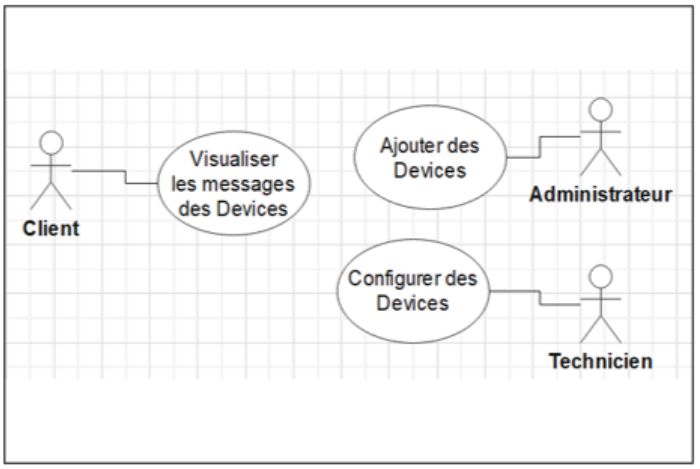

Figure 3 : Interactions entre les utilisateurs et l'application

- **DOSSIER I : GESTION DES DEVICES** *(22 points)*

Sous Dossier 1-1 : STRUCTURE DE DONNÉES *(11 points)*

Un **« Broker »** est un nœud central qui assure l'échange des messages entre plusieurs **Devices**.

Un **« Device »** est un équipement intelligent qui peut envoyer ou recevoir des messages vers ou depuis le Broker. Si le **Device** est un récepteur on le nomme **« Subscriber »** et s'il est un émetteur on le nomme **« Publisher »**. Le diagramme de classes ci-dessous modélise ces objets :

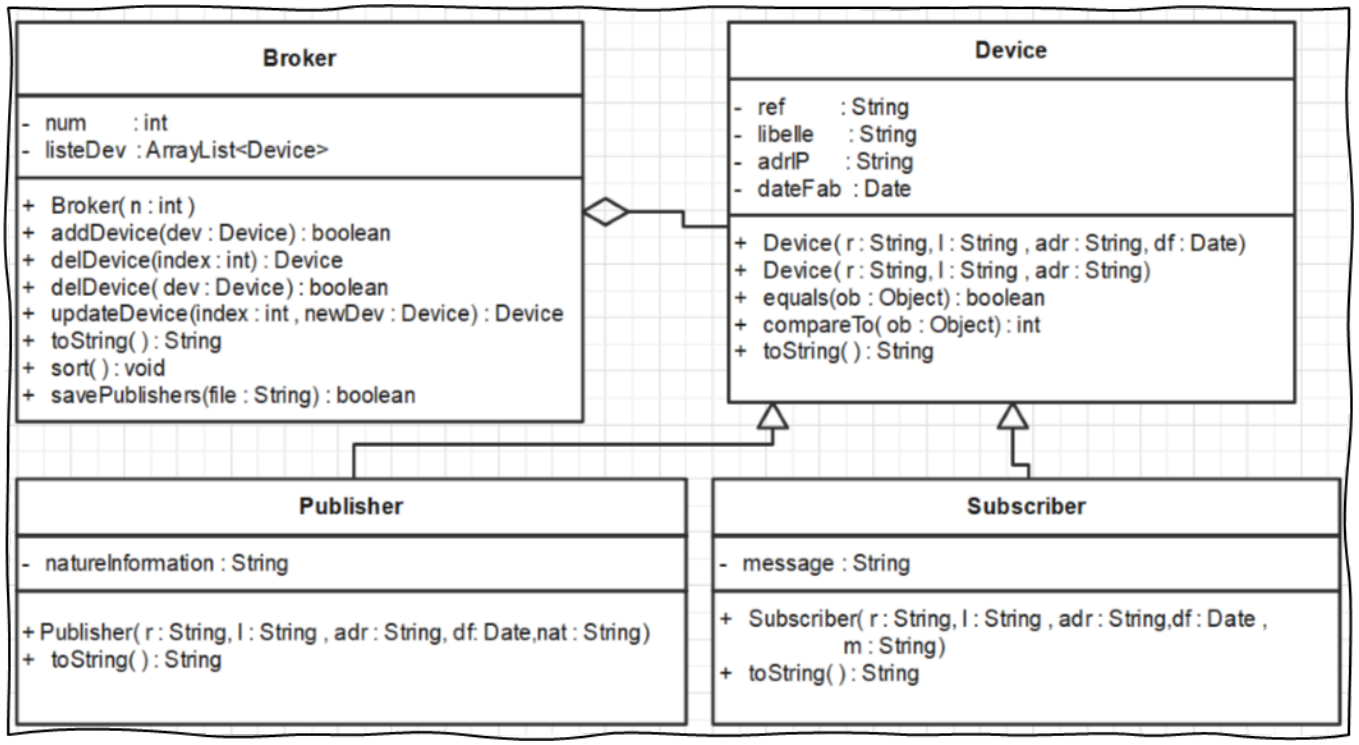

Figure 4 : Diagramme de classes

Implémentation de la classe « Device » :

```
public class Device implements Comparable, Serializable {
   private String ref, libelle, adrIP;
   private  Date dateFab ;
   public Device(...) {...}
   public Device(...) {...}
   @Override public boolean equals(Object ob) { ... }
  @Override public int compareTo(Object ob) { ... }
   @Override public String toString() { ... }
   //Getters & Setters: code non demandé    }
```

✂ - - -

✂ - - -

✂ - - -

Implémentation de la classe « Publisher » :

```
public class Publisher extends Device {
private String natureInformation;
public Publisher(...) { ... }
@Override
public String toString() { ... }
//Getter & Setter: code non demandé
}
```

0.5 pt	**1)** Définir un constructeur de **5 arguments** pour initialiser tous les attributs de cette classe.	réserver à la data matrix de question

1 pt	**2)** Donner la définition de la méthode « **toString** », afin de retourner une chaîne de caractères porteuse d'informations sur un **Publisher** sous-forme : **Référence : xx, Libelle : xx, IP : xx, Date Fab : jj/mm/aaaa** **Nature d'information : xx**	réserver à la data matrix de question

✂ - - -

✂ - - -

public boolean delDevice(Device dev) {

0.5 pt	**5) updateDevice(...)** permet de modifier un **Device** déjà existant et de retourner le **Device** avant sa modification, donner le code adéquat ***(la vérification de la validité de l'index est obligatoire)***	**réserver à la data matrix de question**

public Device updateDevice(int index, Device newDev) {

1 pt	**6) toString ()** retourne une chaîne de caractères porteuse de toutes les informations d'un **Broker** sous-forme : **Numéro de Broker : xx** **Liste des Devices :** **1- Référence : xx, Libellé : xx, IP : xx, Date Fab : jj/mm/aaaa** **Nature d'information : xx** **2- Référence : xx, Libellé : xx, IP : xx, Date Fab : jj/mm/aaaa** **Nature d'information : xx** **3- Référence : xx, Libellé : xx, IP : xx, Date Fab : jj/mm/aaaa** **Message : xx** **...**	**réserver à la data matrix de question**

public String toString( ) {

0.5 pt	**7) sort ()** permet de trier tous les **Devices** de la collection **listeDev** selon leurs références. Donner le code de cette méthode	réserver à la data matrix de question

public void sort( ) {

2 pts	**8) savePublishers (String f)** permet de sauvegarder tous les équipements « **Publishers** » dans un fichier d'objets. Donner le code de cette méthode	réserver à la data matrix de question

public boolean savePublishers(String file) {

- **Sous Dossier 1-2 : ARCHITECTURE CLIENT/SERVEUR** *(11 points)*

Afin que l'administrateur puisse inscrire un Device, une architecture Client/serveur est mise en place. La communication entre le poste de l'administrateur et le Broker se fait par socket TCP, le Broker fonctionne comme serveur d'application. Son schéma est représenté par la figure 5 :

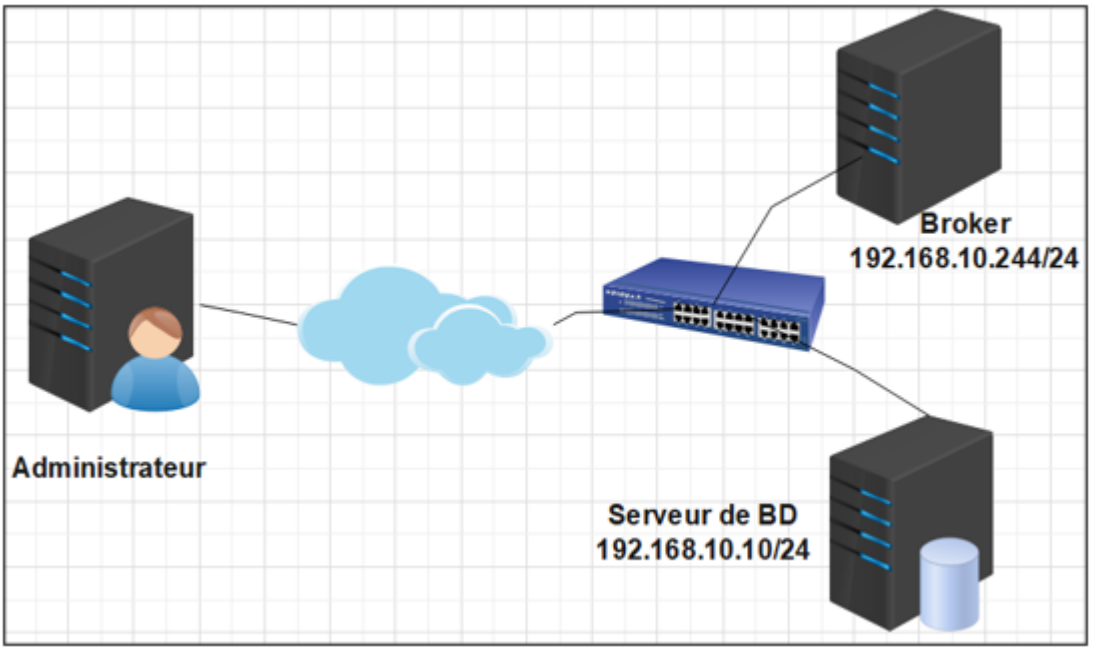

Figure 5 : Architecture client/serveur

Le modèle relationnel et orienté objet d'un **Device** est donné dans la figure 6 :

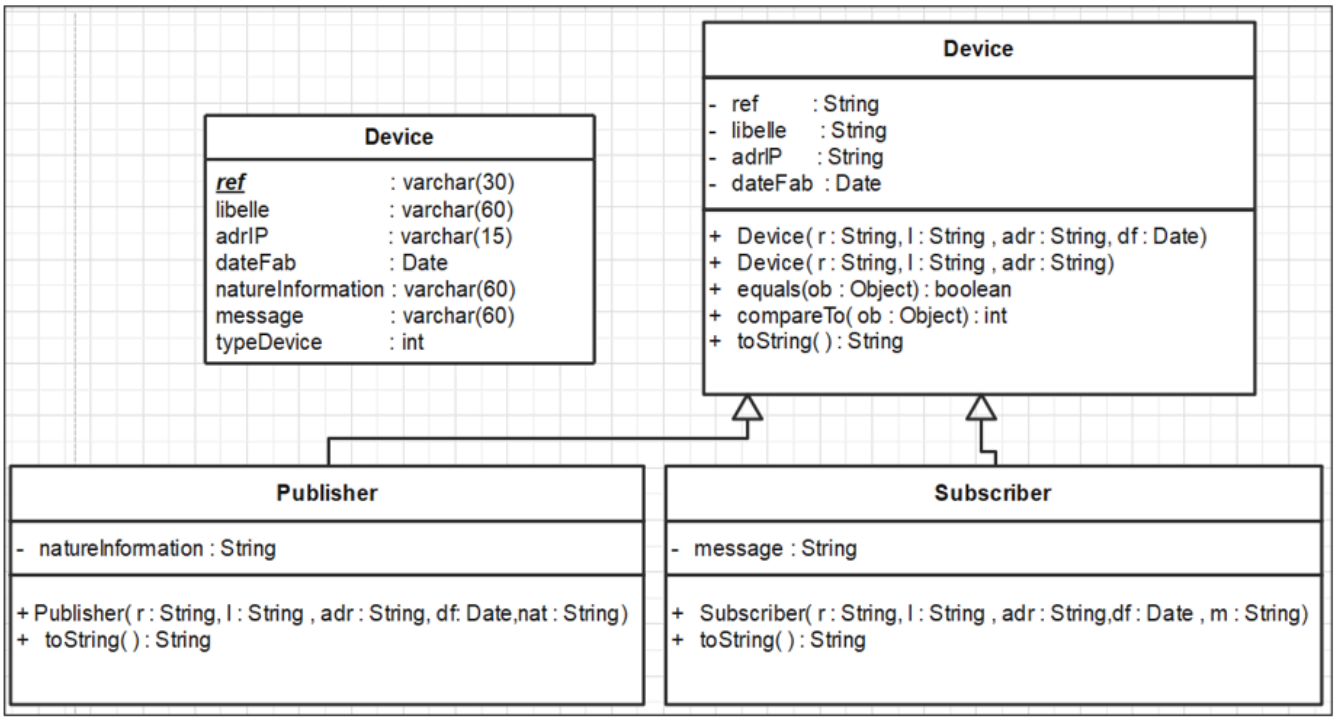

Figure 6 : Modèle relationnel et orienté objet d'un Device

Architecture du serveur :

Les classes à créer dans le serveur d'application sont : **DAO** et **Serveur** (Broker).

Classe « DAO » : permet la connexion à la base de données.

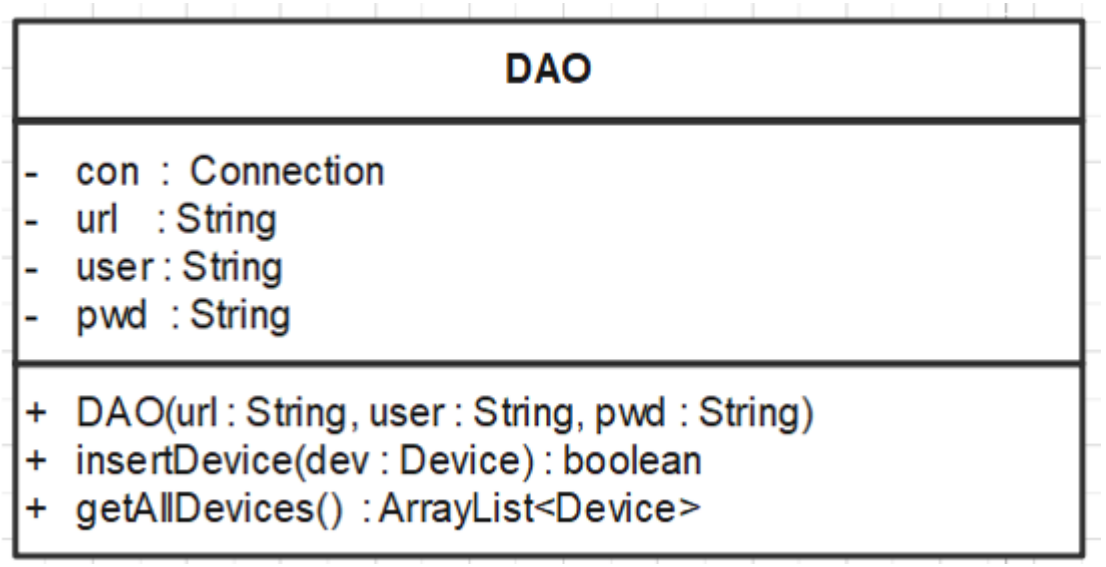

Figure 7 : Classe « DAO »

Elle est constituée de :

- **4** attributs ;
- Un constructeur avec **3** arguments qui initialise les attributs « url », « user » et « pwd » et qui ouvre une connexion avec le serveur de base de données ;
- Une méthode « **insertDevice** », permet l'ajout d'un nouveau Device dans la table « Device » de la base de données. On suppose que la référence du Device est unique, le champ « **typeDevice** » prend la valeur '1' pour un Publisher ou la valeur '2' pour un Subscriber ;

NB : Le code de la méthode « **getAllDevices** » n'est pas demandé.

✂ - - -

public boolean insertDevice(Device dev) {

Classe « Serveur » du serveur :

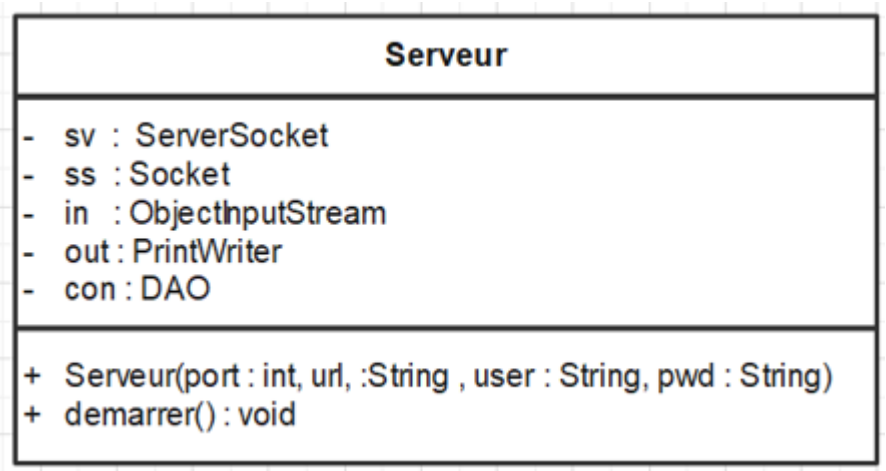

Figure 8 : Classe « Serveur »

```
public class Serveur {
private ServerSocket sv;
private Socket ss;
private PrintWriter out;
private ObjectInputStream in;
private DAO con ;
public Serveur(int port, String URL, String user, String pwd) {
try{
sv = ... [1] (0.5 pt)
con = ... [2] (0.5 pt)
} catch(Exception ex) { ex.printStackTrace() ; }
}
public void demarrer(){
Device dev;
while(true) {
try {
ss= sv.accept();
out = ... [3] (1 pt)
in = ... [4] (1 pt)
do { //lire un Device depuis l'administrateur
dev = ... [5] (1 pt)
if(dev != null) {
con.insertDevice(dev);
out. ... } [6] (1 pt)
else out.println("end");
} while(dev != null);
} catch(IOException ex) {ex.printStackTrace(); break;}
} }
```

✂ - - -

Architecture client :

La classe de cette architecture est :

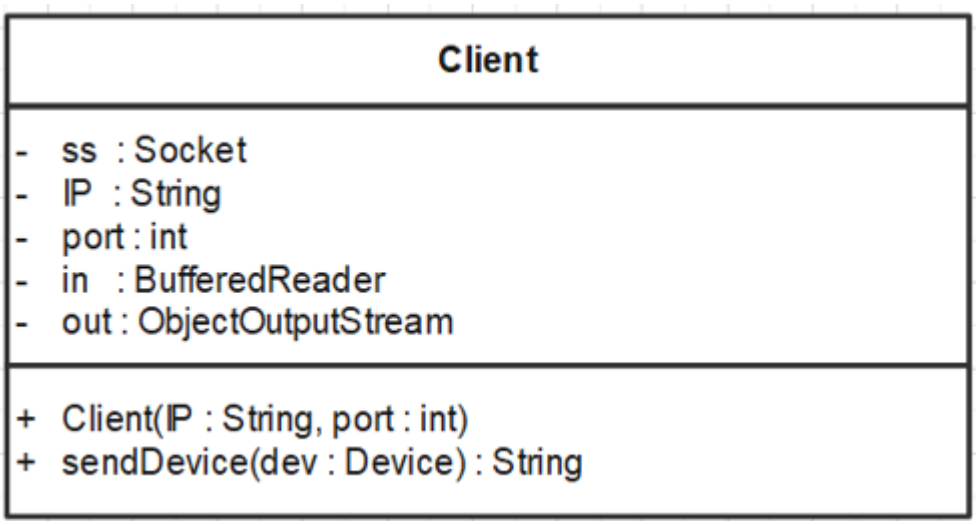

Figure 9 : Classe « Client »

```
public class Client{
private String IP;
private int port ;
private ObjectOutputStream out;
private BufferedReader in;
private Socket ss ;
public Client(...) { ... }
public String sendDevice(Device dev){ ... }
}
```

1.5 pt	**1)** Donner le code du constructeur avec **2** arguments qui initialise les **2** premiers attributs, instancie **Socket** et prépare les objets de communication (**in** et **out**)	réserver à la data matrix de question

1.5 pt	**2)** Donner le code de la méthode **sendDevice()** qui envoie au serveur un **Device** à ajouter et retourne le message de notification renvoyé par le serveur.	réserver à la data matrix de question

public String sendDevice(Device dev) {

DOSSIER II : PLAN DE CONFIGURATION DES DEVICES *(10 points)*

Pour assurer la configuration des **Devices** par un technicien, une application Desktop développée sous Microsoft Visual Basic est mise en place. Les données sont stockées dans une base de données SQL-Server. Les caractéristiques du serveur sont :
- Nom du serveur : **srv_IOT** ; - Nom de la base de données : **db_Device** ;
- Authentification Windows.

Le **MLDR** de cette base de données est le suivant :

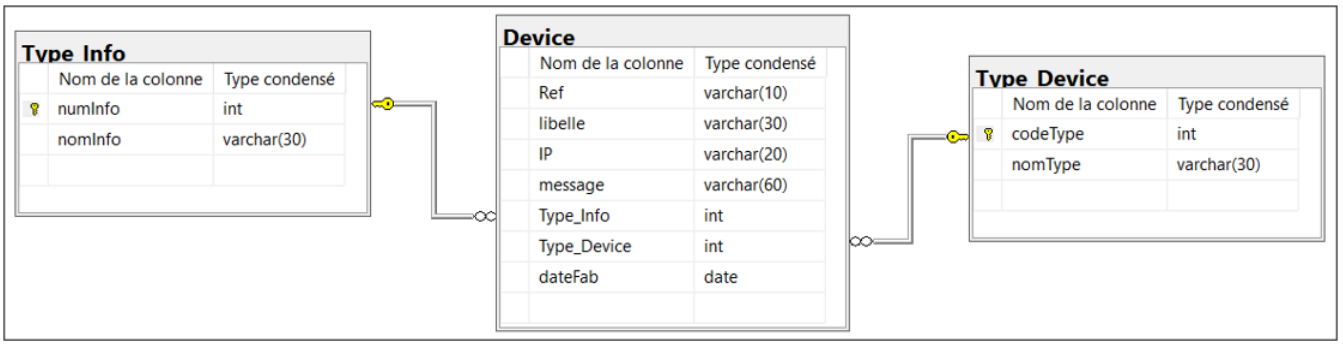

Figure 10 : Modèle logique de données relationnel

Consigne : Choisir le mode de connexion à utiliser (***connecté*** *ou* ***non connecté***).
Toutes les questions doivent être traitées avec le même mode de connexion.

1 pt	**1)** Dans le module principal **Md1.bas**, déclarer les objets et les bibliothèques nécessaires pour se connecter à la base de données.	réserver à la data matrix de question

1.5 pt	**2)** Dans le module principal **Md1.bas**, créer la fonction **« Connecter() »** qui établit une connexion avec le serveur de base de données et retourne un Boolean qui reflète l'état de cette opération	réserver à la data matrix de question

Public Function Connecter() As Boolean

✂ - - -

2 pts	**4)** Écrire le code de la procédure **« lister_DevicesForType(…) »** qui affiche les **Devices** pour un type donné, le nom de type de **Device** voulu est passé comme argument. La liste doit être affichée dans un objet **DataGridView** nommé **« DGV_Device »** comme suit :	réserver à la data matrix de question

	Ref	Libelle	IP	Nom Type info
▶				

Private Sub lister_DevicesForType(ByVal nomType As String)

2 pts	**5)** Donner le code de la fonction **« IP_Exist(…) »** qui prend en argument une adresse IP et retourne : - **« true »** si elle est déjà attribuée à un Device ; - **« false »** dans le cas contraire.	**réserver à la data matrix de question**

Private Function IP_Exist(ByVal IP As String) As Boolean

2 pts	**6)** Écrire le code de la procédure **« Attribuer_IP(…) »** qui permet d'attribuer une adresse IP à un **Device**, la procédure prend comme argument la référence et l'adresse IP à attribuer. **NB :** Les 2 vérifications suivantes doivent être faites avant l'attribution : • La référence doit être existante ; • L'adresse IP doit être unique. Si l'une de ces conditions n'est pas vérifiée, un message de notification est nécessaire.	**réserver à la data matrix de question**

Private Sub Attribuer_IP(ByVal ref As String, ByVal IP As String)

- **DOSSIER III : MONITORING DES MESSAGES DES DEVICES** *(8 points)*

Pour visualiser les messages des Devices d'un type donné, le client utilise une application Web. Les enregistrements sont stockés dans la base de données « db_Device » implémentée sous MYSQL dont une partie de son MLDR est donnée dans la figure 11.

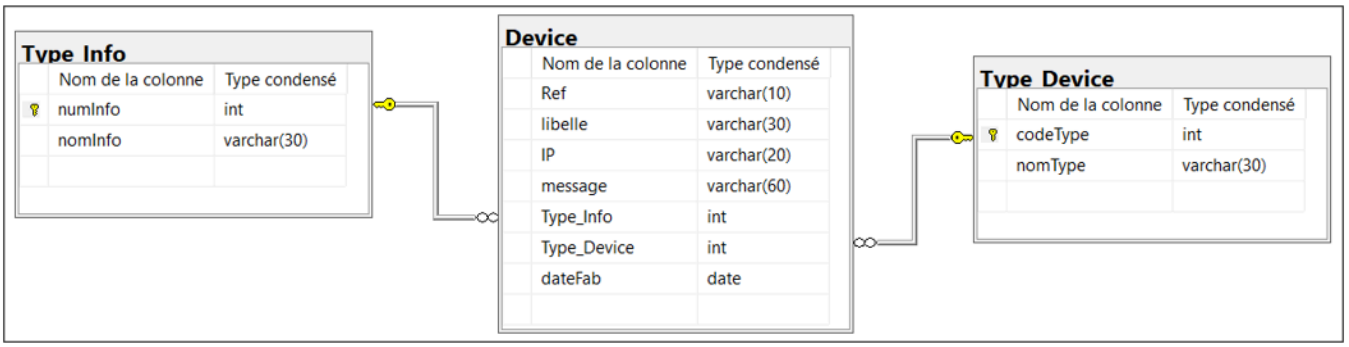

Figure 11 : Partie du Modèle logique de données relationnel

- Nom du serveur : **srv_Device** ; - Nom de la base de données : **db_Device** ;

- Nom utilisateur : « **user_iot** » - Mot de passe : **user@iot@**

Consigne : Vous devez choisir un seul mode (procédural ou orienté objet).

2 pts	**1)** Écrire le code PHP du fichier **« connecter.php »** permettant la connexion à la base de données **« db_Device »**.	réserver à la data matrix de question

// Code du fichier **« connecter.php »**

.....

.....

.....

.....

.....

.....

La page web **« chooseTypeDevice.php »** permettant de choisir un type de Device est donnée dans la figure 12 :

Figure 12 : Choix d'un type Device

Le code de cette page est le suivant :

```
<!DOCTYPE html>
<html>
<head> <title>IOT</title> </head>
<body>
<h1>monitoring messages</h1>
<form method="post" action="showMes.php">
<label>Choose Type of Device</label>
<select name="typeDev">
```

✂ - - -

Le clic sur le bouton de validation **« Show Message »** permet d'ouvrir la page du fichier **« showMes.php »** illustrée par la figure 13 :

Message Of Type : Inductif Sensor

Message	Nom Information	Libellé
14 g	Poids	Capteur de poids
18 cm	Distance	Capteur de distance
25 Tesla	Champ magnétique	Capteur de champ

Figure 13 : Listing des messages d'un type Device

Le code de cette page est le suivant :

```
<!DOCTYPE html> <html> <head> <title>show</title> </head>
<body> <?php
require("connecter.php") ;
function getNameTypeDevice($ref) [1]
{ ... }
?>
<h1>Messages of type : <?= getNameTypeDevice(...)?> [2] </h1>
<table> <thead> <tr><th>Message</th><th>Nom information</th>
<th>Libelle</th> </tr> </thead>
<tbody>
<?php // [3] lister : message, Libelle et nomInfo du type de Device choisi
...
?>
</tbody>
</table>
</div>
</body> </html>
```

✂ - - -

✂ - - -